

Accelerating LLM Serving for Multi-turn Dialogues with Efficient Resource Management

Jinwoo Jeong
Korea University
Seoul, Korea

Jeongseob Ahn
Korea University
Seoul, Korea

Abstract

Although there have been significant efforts to make LLM serving efficient, we observe two limitations of current state-of-the-art serving frameworks in handling multi-turn dialogues between users and assistants, particularly in chat scenarios. First, existing LLM frameworks incur substantial computational overhead in recomputing attention keys and values (KV) for understanding context across multiple turns of user queries. Second, as the prompt length of user queries is amplified due to multi-turns, a first-come-first-served (FCFS) scheduling policy often causes head-of-line blocking issues, leading to underutilization of GPU resources.

To address these limitations, we present *FlashGen* to rapidly complete multi-turn queries by efficiently utilizing the compute and memory resources of GPUs as well as the host hardware (e.g., DRAM and SSD). We introduce a multi-level KV cache comprised of GPU, CPU, and SSD, to efficiently retain attention KVs from prior turns. Our approach employs low-cost cache restoration techniques to avoid the recomputation burden. Further, we propose a request reordering technique to effectively utilize GPU memory. This scheduling technique carefully adjusts the request order without compromising fairness. Our proposed techniques outperform the vLLM framework in terms of both latency and throughput. For OPT 30B and Llama-2 70B models with the ShareGPT dataset, we achieve 1.63× and 2.85× better throughput, respectively while in a similar latency boundary.

CCS Concepts: • Computer systems organization; • Software and its engineering → Memory management; Scheduling;

Keywords: LLM Serving; Multi-turn Dialogues; KV Cache Management; Request Reordering

ACM Reference Format:

Jinwoo Jeong and Jeongseob Ahn. 2025. Accelerating LLM Serving for Multi-turn Dialogues with Efficient Resource Management. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*,



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.

ASPLOS '25, Rotterdam, Netherlands

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1079-7/25/03

<https://doi.org/10.1145/3676641.3716245>

Volume 2 (ASPLOS '25), March 30-April 3, 2025, Rotterdam, Netherlands. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3676641.3716245>

1 Introduction

As transformer-based large language models (LLMs) continue to gain significant attention in various domains, it becomes increasingly important to deal with longer context sizes, such as sustained conversations with a chatbot or summarizing long passages [7, 16, 32]. For instance, GPT4-Turbo provides up to a 128K context [24]. Although there have been several efforts to reduce the cost of serving LLMs [17, 37], the demand for serving long contexts (e.g., chatbots) exacerbates the challenges of providing high computational capability and memory capacity with GPUs.

Typically, a chatbot conversation session involves a multi-turn dialogue where users can ask follow-up questions and receive relevant responses, as shown in Figure 1. To ensure high-quality answers to users, it is crucial for chatbots to retain the context of previous dialogue turns. For transformer-based generative models, this context information is represented by the attention keys and values (KV) associated with the prompt and generation of previous turns. The attention mechanism is a key technique of transformer-based LLMs, which captures contextual information within the input sequence [34]. The model processes each input token while considering all other tokens in the sequence, assigning different attention weights to each token based on its relevance to the current token.

However, due to the limited memory capacity of modern GPUs, caching all the KV of the attention layers used in previous chat turns becomes prohibitively expensive [29]. Instead, current LLM frameworks such as vLLM and TensorRT-LLM take an approach that recomputes previous turns. To generate a response, this approach combines the chat history with the current input to form a prompt [11, 15, 17]. While this can reduce memory consumption, it renders serving systems inefficient in making forward progress.

This paper reveals that state-of-the-art LLM serving frameworks are inefficient at handling multi-turn prompts used in conversational AI. We conduct a performance characterization study when serving an OPT 13B model with a real-world conversational dataset, ShareGPT [27]. We highlight two key findings. First, as the number of turns in a chat session increases, the prompt length proportionally increases to

incorporate previous turns. We call this the prompt amplification problem. Upon scheduling such multi-turn prompts, it takes a considerable amount of time to recompute the attention KVs associated with previous turns. This performance overhead stems from the lack of context retention in GPU memory. Given the increasing trend of longer prompts and multi-turn scenarios, we need to revisit the current approach.

Second, the amplification of prompt lengths in multi-turn scenarios poses challenges in achieving cost-effective GPU memory utilization. As the prompt length increases, the required memory space also grows proportionally. When the memory space is insufficient to accommodate the prompt at the head of the queue, the first-come-first-served (FCFS) scheduling policy employed in modern LLM serving schedulers [15, 17, 37] leaves the remaining GPU memory space idle. While such idle space can be utilized in caching KVs of completed previous turns, we observe that the effectiveness of GPU caching decreases due to excessive cache contention under high loads. To effectively utilize the limited GPU memory at a diverse range of workloads, we need a more efficient resource scheduling strategy.

To this end, we introduce *FlashGen* to expedite the completion of multi-turn prompts by efficiently utilizing the compute and memory resources of GPUs and the host hardware (e.g., DRAM and SSD). First, we design a multi-level cache for attention KVs, called *FlashGen-Cache*, to efficiently retain context in multi-turn scenarios by leveraging GPU memory, host memory, and SSD. The key observation is that it is more efficient to store computed attention KVs of previous turns in host memory rather than to recompute them. Since our KV manager can minimize the latency of restoring attention KVs from SSD to GPU via host memory, we can achieve significant performance benefits over recomputation.

Second, we present a request reordering technique called *FlashGen-Sched*, designed to effectively utilize the GPU memory under increasing request loads. As the request load increases, the GPU cache hit rate steadily decreases due to the limited capacity. So, we repurpose the idle memory space to execute awaiting requests. Our scheduler allows younger requests to be dispatched ahead of older ones in cases where older requests are not runnable due to insufficient free memory. While this simple technique improves overall performance, it may lead to a starvation problem for requests conceding their turns. To address this, our scheduler preempts the promoted requests when the combined memory space of these requests and the remaining free space becomes sufficient to accommodate the request yielding its turn.

We implement our *FlashGen* on top of a popular serving framework, vLLM, which incorporates recent advances such as PagedAttention [17] and iteration-level scheduling [37]. We evaluate the effectiveness of our techniques with diverse datasets including ShareGPT [27], which is a real-world chatbot service, as well as Alpaca [31] and HumanEval [8]. We use an Azure instance, Standard_NC48ads_A100_v4, which

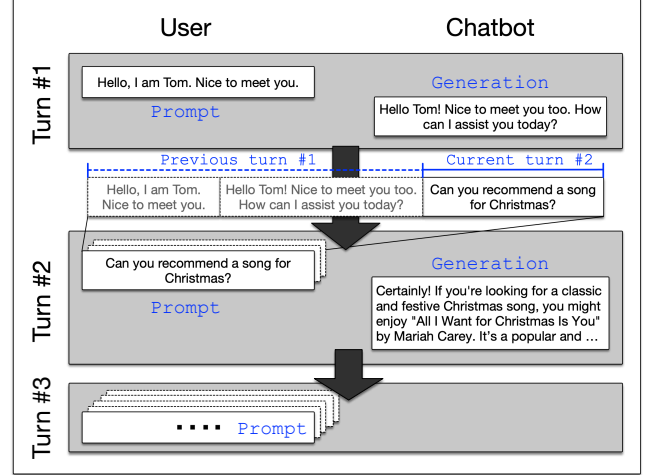


Figure 1. An example of a multi-turn conversation

is equipped with two NVIDIA A100 (80GB) GPUs and 440GB of host memory. For archiving KVs, we use two NVMe SSDs (2 x 960GB) given in the instance. For the ShareGPT dataset, FlashGen can achieve 1.63× and 2.85× better throughput on OPT 30B and Llama-2 70B models, respectively while in the same level of latency, compared to vLLM.

The new contributions of this paper are as follows.

- This paper analyzes the performance inefficiencies in serving multi-turn prompts used in conversational AI and identifies two distinct problems: prompt amplification and recomputation.
- This paper designs a multi-level attention KV caching technique by leveraging GPU memory, host memory, and storage and shows how our technique can efficiently deal with multi-turns dialogues.
- This paper employs a request reordering technique to maximize GPU memory utilization while preventing the starvation problem among requests.

2 Background and Motivation

2.1 Inference of Generative Models

Transformer-based generative models (e.g., GPT) have become the de facto standard in the era of generative artificial intelligence. There are two distinct characteristics in inferences of the generative large language models (LLMs). First, LLM inferences operate in an autoregressive manner [34], involving two distinct phases. Initially, it takes an input sequence of tokens, known as a prompt, and generates an initial output token. This step is commonly referred to as the *prompt (or prefill) phase*. Subsequently, the generated output token is fed back into the model to generate the next output token. We call this the *generation phase*. This generation phase is *repeated* until it produces an end-of-sentence (EOS) token or reaches the model's maximum token limit.

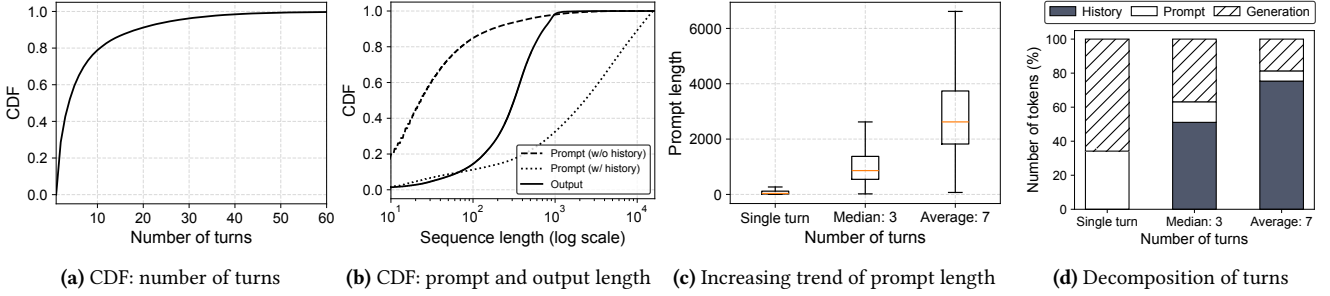


Figure 2. Multi-turn characteristics of a real-world chatbot dataset: ShareGPT [27]

Second, generative models take advantage of the attention mechanism, which computes the correlation among tokens, encompassing both prompt and generated tokens [34]. This mechanism helps the model in determining the relative importance of different tokens, ensuring that the generated text remains contextually relevant. The attention layers retain the states of keys and values (KVs) for all tokens encountered so far, eliminating the need to recompute information for preceding tokens in subsequent iterations. Caching KVs is a common technique for speeding up LLM inferences [1, 4, 17, 20, 33, 37].

2.2 Transformer-based Chatbots

Transformer-based chatbots, such as OpenAI’s ChatGPT [25] and Google’s Bard [13], represent a significant leap in the field of conversational AI and have garnered widespread attention for their impressive capabilities. Users interact with these chatbots by making an inference request and receiving responses generated by pre-trained language models. A conversation comprises multiple rounds of inference requests and responses. In this continuous dialogue, chatbots are required to maintain contextual information about the entire interaction to ensure contextually relevant responses.

To maintain the context of previous turns within the same conversation session, a chatbot needs access to the attention KVs of all tokens associated with previously exchanged conversations. We call this *history KVs*. However, retaining all these attention KVs in GPU memory throughout the conversation session is prohibitively expensive due to the lack of GPU memory capacity [3, 28, 29]. To overcome this challenge, modern LLM frameworks employ a practical workaround. Instead of keeping attention KVs of prior chat turns in GPU memory, the client provides the current input prompt and the output tokens generated in prior turns together [11, 15, 17]. Figure 1 presents that the input sequence (prompt) of the second turn actually contains the prompt and output of the first turn. Similarly, the third turn contains the tokens from the first and second turns. The chatbot then computes the combined input, eliminating the need to store attention information of prior turns in GPU memory.

2.3 Characteristics in Multi-turn Dialogues

To understand the distinct characteristics of multi-turn dialogues, we analyze the ShareGPT [27] dataset, which comes from a real-world chatbot service. Our experiments are conducted with an OPT-13B model [38] on a single A100 GPU. The detailed experimental setup is presented in Section 5.1.

2.3.1 Turns, Prompts, and Generations in a Multi-turn Dataset.

Figure 2a presents the cumulative distribution function (CDF) of the number of turns per session. Due to the limited space, we show up to 60 turns. Note that chat sessions that are more than 60 turns take only 0.3% in the ShareGPT dataset. We observe the median and average number of turns per chat session are 3 and 7, respectively. The number of single-turn cases occupies only 28% while more than three turns shows 48% of the total number of sessions.

Figure 2b shows the CDF of the number of tokens provided by prompts and those generated during generation phases, respectively. We divide the prompts into two cases: one with history tokens included and the other without. On median, the prompt length without history is 23, while the length with history becomes 2,294 (99× longer). This substantial difference in prompt length suggests a significant contribution to the computational cost when considering prior chat turns.

Figure 2c compares prompt lengths across three cases: single turn, median turns, and average turns. As the number of turns increases, the prompt length expands significantly. For each case, we decompose the token sources into three segments: history (previous turns), prompt (current turn), and generation. Figure 2d illustrates the average for each case. Except for the single turn cases, the recomputation stemming from history takes a significant portion of the inputs. Even in the median case, the tokens attributed to history comprise more than half of the total.

In multi-turn dialogues, the total prompt length is significantly amplified by its prior turns, known as *history*, as the number of turns increases. However, since existing LLM serving frameworks [15, 17, 33] do not take into account

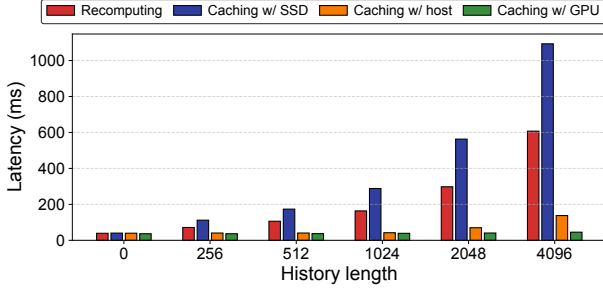


Figure 3. Latency of processing prompt with history (prior turns) in a microbenchmark

	OPT-13B	OPT-30B	OPT-66B	OPT-175B
History KVs (GB)	2.8	4.7	8.1	16.2

Table 1. The average size for history KVs per session when serving OPT models with the ShareGPT dataset

these multi-turn characteristics, the performance efficiency is limited when serving conversational AI.

2.3.2 Cost of Handling Multi-turn Prompts. As preserving history KVs on the GPU incurs significant overhead in terms of space, current LLM serving frameworks spend additional time recomputing tokens of prior turns. To address this issue, we conduct a preliminary study of caching history KVs in the host memory and make a comparison with GPU caching as an ideal case.

Figure 3 illustrates the comparison of latency in processing prompts with varying history lengths from 0 to 4,096, while maintaining a fixed prompt length of 256. The baseline approach, which recomputes KVs of prior turns without caching them on either the GPU or host memory, is represented by the first bar. With caching history KVs in either SSD (second bar) or host memory (third bar), the cost of computing KVs of prior turns is replaced by the cost of transferring those KVs from SSD to GPU and the host to GPU memory, respectively. Due to the limited bandwidth of SSD, retrieving history KVs directly from SSD is generally undesirable. However, leveraging host memory can significantly reduce the recomputation cost. Ideally, with KVs kept in GPU memory (fourth bar), requests can be served without recomputation or transmission. As the history length increases, the performance gap between recomputation and caching methods becomes significant. The latency of host caching slightly increases from a 2K history length due to the transmission overhead, while the GPU caching maintains consistent performance, demonstrating that caching KVs computed in previous turns is more efficient than recomputation.

However, due to limited GPU memory capacity, caching KVs of previous turns on the GPU is not feasible. As the request load increases, the GPU memory quickly fills up

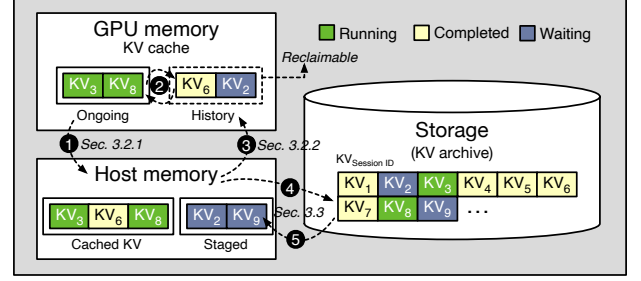


Figure 4. Multi-level attention KV cache comprised of GPU memory, host memory, and storage

with KVs generated during ongoing requests. Consequently, the remaining space for preserving history KVs is significantly reduced at high loads. Table 1 shows the average size for history KVs per session in the ShareGPT dataset when serving diverse OPT models formatted FP-16. The substantial memory consumption of history KVs indicates that relying solely on GPU or host memory would hinder the scalability of concurrent sessions (or loads). To tackle this limitation, we explore a multi-level cache design comprised of GPU memory, host memory, and storage.

3 Managing Attention Keys and Values

In this section, we introduce our attention KV cache, called FlashGen-Cache, designed to avoid the cost of redundant computation in processing multi-turn dialogues. Since multi-turn prompts consume a large amount of memory space, it is important for LLM serving frameworks to complete them rapidly, thereby increasing resource efficiency. We describe our caching design with GPU memory, host memory, and storage in the following.

3.1 Design Overview

Figure 4 presents our multi-level caching hierarchy designed to manage attention KVs from prior turns within our LLM serving framework. We design a two-stage data path: (1) between GPU and host memory, and (2) between host memory and the storage. The primary objective of our FlashGen-Cache design is to minimize *the number of transmissions* and *the transmission latency* for KVs between GPU and host memory.

① During inferences, we proactively make a copy of the generated KVs (denoted as *Running*) into the host memory to minimize the overhead associated with reclaiming GPU memory. ② Even after completing the execution of an inference request (denoted as *Completed*), we retain the generated KVs on the GPU memory as cache, but mark them as *reclaimable*. This reclaimable space is considered a free region because the corresponding KVs are already cached in the host memory. ③ Upon a GPU cache miss, we transmit the corresponding KVs on a layer-by-layer basis to enable the immediate execution of layers whose KVs are completely

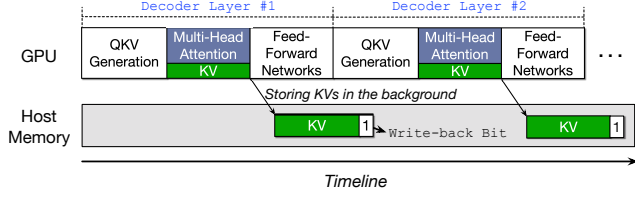


Figure 5. Workflow of caching generated KV pairs in the host memory during inference (The write-back bit indicates that the KV needs to be stored in the storage)

loaded. This strategy effectively overlaps the transfer of KV pairs with layer computation.

With the growing trend of longer sequences, the host memory would not be sufficient to cache all the previous attention KV pairs as discussed in Section 2.3.2. To address this limitation, we extend our caching capabilities with the integration of SSDs. Since modern SSDs are plentiful and cheap compared to memory, this is a cost-effective approach to expand the caching capacity. ④ To mitigate the reclamation cost from host memory, we periodically make a copy of the KV pairs into SSDs asynchronously in the background. Note that this operation is not on the critical path. However, a challenge arises in the latency associated with loading cached KV pairs from SSDs to GPU memory directly due to the limited storage bandwidth. To tackle this challenge, ⑤ we allocate a portion of the host memory to proactively stage the requested KV pairs from SSDs to host memory upon the arrival of user queries at the server (denoted as *Waiting*), thus alleviating the performance impact of loading KV pairs from SSDs.

In summary, for caching attention KV pairs, there are three possible configurations: GPU-only, GPU+CPU, and GPU+CPU+SSD. We describe each caching layer in the following subsections.

3.2 Caching KV pairs in GPU and Host DRAM

The current LLM frameworks primarily utilize GPU memory to cache attention KV pairs only for ongoing requests. So, our GPU caching strategy (GPU-only) aims to efficiently utilize the remaining space to cache KV pairs from previous turns in a best-effort manner. Even after a given request (or turn) is completed, we retain its attention KV pairs in the GPU memory as cache. Later, if a request arrives and its KV pairs for previous turns are still cached in the GPU memory, we can immediately serve the KV pairs without the recomputation phase for the previous turns. Upon such a cache hit, the corresponding space is no longer reclaimable because it is being used for the currently running request.

Meanwhile, we mark that space used for caching KV pairs of completed requests as *reclaimable*, allowing the running requests to make use of the space if needed. When GPU memory is insufficient, we can rapidly reclaim the space by discarding the oldest KV pairs in the completed requests in a FIFO manner. On a cache miss, our GPU-only design needs to recompute its KV pairs of previous turns like the baseline.

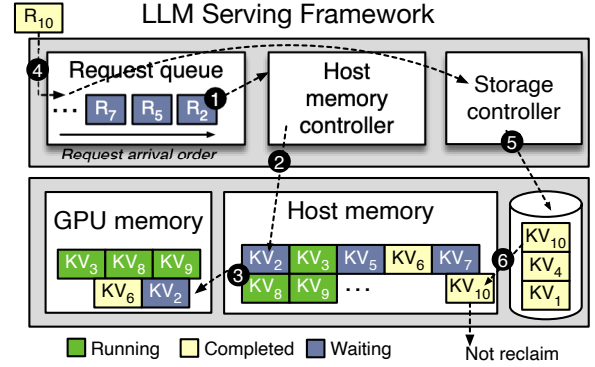


Figure 6. Overview of managing attention KV pairs with our multi-level cache hierarchy

While caching history KV pairs in GPU is a simple yet effective approach, relying solely on GPU caching is ineffective for reducing the recomputation cost due to limited GPU memory capacity, leading to excessive GPU cache misses. Thus, we extend our caching capabilities with the host memory, called GPU+CPU. By caching the generated KV pairs from previous turns in the host memory, we further reduce the need for a recomputation phase when a request arrives from the same chat session later. However, this host caching approach introduces additional time for storing and retrieving the KV pairs belonging to its previous turns to and from host memory. Thus, it is imperative to minimize this overhead.

3.2.1 Preserving KV pairs in Host DRAM. Unlike conventional CPU caches, which typically evict cache lines upon encountering cache misses, we adopt a different approach by immediately copying attention KV pairs to host memory once they are generated during inferences. Figure 5 presents that the generated KV pairs are written into the host memory while performing the inference execution. Since these generated KV pairs remain unchanged during the inferences, we gradually copy the newly generated KV pairs to the host memory at each attention layer asynchronously. Our strategy helps minimize the replacement overhead from the GPU to host memory by hiding the eviction latency with subsequent computations. In addition, this simplifies the reclaim procedure for GPU memory, as the KV pairs for completed requests are already copied into the host memory.

3.2.2 Reloading KV pairs from Host DRAM to GPU. Figure 6 presents how we deal with cached KV pairs in the host memory. ① When a request arrives at the serving framework, we first check whether the corresponding KV pairs from previous turns are cached in the GPU memory. Suppose a GPU cache miss occurs. Then, ② we need to look up the KV pairs in the host memory which is our second-level cache. ③ If found, we restore them to GPU memory. When the request cannot be scheduled immediately due to precedent

requests, we defer the transmission for the cached KVs until the pending requests are completed. In the meantime, we prevent those KVs from being reclaimed. If we cannot find the matched KVs in the host memory, we request the lower caching layer (here in SSD) to retrieve them.

When restoring KVs from host memory, we leverage the pipeline approach [6] to minimize the idleness of GPU compute units. While executing the first transformer layer of the model, we transmit the required KVs for the second layer simultaneously. This optimization minimizes the miss penalty by hiding the latency of the KV restoration from the host memory to the GPU memory.

3.2.3 Batch-aware KV Restoration. In the iteration-level scheduling, requests in a batch can be composed of a combination of prompt and generation phases. If a batch contains one or more prompt phases, the amount of computation is sufficient to hide the transmission of attention KVs when restoring KVs from host memory. On the other hand, when a batch consists solely of generation phases, its computation is relatively less because the number of tokens to be computed is small. Consequently, even with our restoration technique, the execution is frequently suspended to wait for the completion of transferring the required KVs. This impedes the forward progress of the batch, leading to a slowing down of other requests in the same batch. To avoid this problem, we have the scheduler make a batch of remaining requests in the pool by excluding that request whose KVs are not completely loaded. This approach ensures that the currently running requests are not adversely affected. Once all required KVs are ready, we immediately compose a new batch by including the request. This optimization reduces the waiting time for the request to join the batch while not harming the execution of other requests.

3.2.4 Discussion: proactive (inclusive) vs. lazy (exclusive) caching. Our proactive caching aims to minimize the performance overhead associated with reclaiming KVs from GPU to CPU memory. When the GPU memory becomes insufficient, this proactive approach allows us to immediately serve incoming requests by simply discarding the KVs in the GPU memory. That is a key advantage of the inclusive cache. Also, we do not observe that the KV transmission negatively affects the iteration time (See Section 5.3), as the amount of KVs newly generated per layer in each iteration is not significant. Conversely, the lazy approach can effectively utilize the host memory space by deferring the reclamation, working as an exclusive cache. However, when serving requests for long contexts, it causes the direct reclamation procedure, leading to latency increases in the tail.

3.3 Expanding Caching Space for KVs with SSDs

We extend our caching design to archive KVs generated from previous turns in SSD (denoted as GPU+CPU+SSD). This design decision is made by insufficient memory capacity even with

the host memory to preserve all prior KVs. As discussed earlier in Section 2.3.2, the volume of generated KVs expands proportionally with the increase in the number of sessions.

By adhering to the inclusion property for caching KVs in host memory, we can efficiently archive the generated KVs on SSD at a minimal cost. We follow the same design principle used for preserving KVs from the GPU to host memory. Specifically, we periodically write a copy of the cached KVs from the host memory to the storage device asynchronously in the background. Due to the limited bandwidth of modern SSDs, we do not directly write these KVs from GPU to SSD.

When restoring cached KVs from SSD, we follow a two-step process. Suppose a request arrives, but the corresponding KVs are not cached in the host memory (R_{10} of Figure 6). As shown in Figure 6 (4-6), our KV manager initiates the restore procedure, transferring the archived KVs from storage to host memory. Although it is possible to supply the KVs directly from storage to GPU using GPUDirect [21, 22], this is not preferred due to the limited read bandwidth of SSDs. Instead, we place them in the host memory (denoted as *staged* in Figure 4) and mark them as non-reclaimable, since these KVs will be used shortly. Once the scheduler dispatches the request, our KV manager copies the KVs from the host memory to the GPU, as described earlier. Such a staging strategy can hide the restoration time if there are pending requests ahead. In other words, if there is no opportunity to hide the restoration phase (i.e., when the number of waiting requests is less than 1), it causes delays in scheduling until its history KV is completely loaded. In that case, we opt for recomputation as a fallback mechanism because it is faster than retrieving the KVs from SSDs due to the limited bandwidth of SSDs as shown in Figure 3.

Meanwhile, under high demand, staging KVs in the host memory can become a capacity burden. Therefore, we provide a control knob to adjust the host memory usage for staging KVs. By default, the staging space is configured to accommodate the maximum sequence length of the models, preventing a single request from failing to be staged due to lack of space. If the staging region is fully occupied, our KV manager defers loading the KVs even though the request is queued. Once the memory space becomes available by proceeding with requests in the queue, the KVs corresponding to the deferred requests are loaded.

4 Effective GPU Memory Utilization with Scheduling

This section introduces our FlashGen-Sched to effectively utilize the remaining GPU memory after allocating for ongoing requests. We first explain why the GPU memory is not fully utilized and then discuss why caching KVs of completed requests in GPU can be inadequate for effectively increasing memory utilization at high loads. Finally, we present our scheduling technique, which opportunistically utilizes

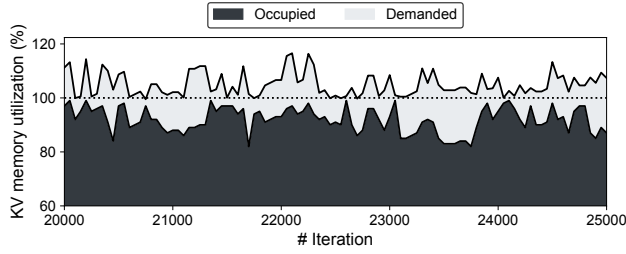


Figure 7. GPU (2 x A100-80GB) memory utilization while serving an OPT 30B model with the ShareGPT dataset

idle GPU memory by adjusting the execution order while ensuring fairness among requests.

4.1 Underutilization due to Head-of-Line Blocking

If the available memory space is insufficient to process a given input prompt, the current scheduler in LLM serving frameworks stops processing the input until the memory space becomes sufficient. During this waiting period, the remaining memory space is not utilized. Consequently, even relatively short prompts in the queue that could fit into the current free space are not scheduled. This phenomenon is referred to as the *head-of-line blocking* problem.

Figure 7 exhibits changes in GPU memory utilization as the iteration-level scheduling [37] proceeds. We extract the memory utilization of the KV space at a rate of 0.5 requests per second where the baseline is saturated. The *occupied* region (dark) is utilized for caching KVs of ongoing requests while the *demand* region (light) indicates the amount of memory space required for serving the request at the head of the queue. Except for the occupied region, the remaining space is underutilized because the head request in the queue cannot fit in the current memory space. Although the PagedAttention mechanism of vLLM can effectively increase memory utilization, it is still around 88% on average due to long sequences.

4.2 Towards Effective Memory Utilization

As explored in Section 3.2, we can utilize the idle GPU memory by caching KVs of completed requests. Under high demand of requests, however, caching history KVs in GPU memory is not effective in terms of utilizing the remaining space. This is because the cache hit rate proportionally decreases as request demand increases¹. There are two main reasons. First, the higher load typically means more concurrent users, leading to the contention of GPU memory. To handle the increased number of ongoing requests, the available GPU memory for caching shrinks. Second, in multi-turn dialogues, users interact with agents (e.g., chatbots) and spend time reading responses and typing the next message. As the intervals between turns are not short due to humans

involved, it is challenging to exploit temporal locality in the GPU cache.

Conversely, at low demand, the space is effectively utilized for caching as before. Therefore, we opportunistically reclaim the space used for caching to execute awaiting requests using our reordering technique, while preserving the caching functionality. If the request load is not significant, our reordering technique is not activated because it is unlikely to have pending requests due to a lack of free memory. In such cases, the caching space is not reclaimed by our scheduler and is utilized for keeping KVs as previously. In the following, we explain our request reordering technique.

4.3 Reordering Execution

The state-of-the-art iteration-level scheduler is capable of selecting as many requests from the request pool as the memory space can serve. As shown in Figure 8a, when selecting requests, most LLM serving frameworks [15, 17, 33, 37] consider the order of requests to maintain the first-come-first-serve (FCFS) property. If there is not enough memory to serve the oldest one (R_3 in Figure 8a) in the request queue, the subsequent requests (R_4 and R_5) cannot be scheduled because of the order of the requests, regardless of whether they are runnable. This increases the waiting time for requests while lowering memory utilization, which is the most valuable resource in GPUs. Note that the longer the prompt length of requests, the more memory space is required.

Our proposed scheduler helps to address this phenomenon by fetching runnable requests first, rather than the order of requests. Figure 8b presents how our simple approach can maximize resource utilization. Instead of waiting for the memory space to become sufficient for R_3 , we search for the next runnable request in a greedy manner. In this example, ① we fetch R_4 , which fits on memory. We call R_4 a *promoted* request while R_3 is a *deferred* request yielding its turn. ② By squeezing the idle memory space, we reduce the waiting time for requests in the queue. If the prompt length for R_4 is short and generates a small number of output tokens, its execution can be completed before the other older requests (R_1 and R_2). In such a case, we can utilize the slack time without any fairness issues due to the reordering of requests.

4.4 Starvation-free Scheduling

Our reordering strategy may lead to a starvation problem where requests with high memory demands (e.g., long prompts) are not continuously selected. To address this concern, our scheduler is designed to dispatch deferred requests by preempting promoted requests. We extend the GPU memory manager of the framework to keep track of the memory occupied by promoted requests and treat the space as free memory. Once any of the preceding requests are completed, our scheduler examines the available memory space, including the space occupied by the promoted requests. This allows

¹The experimental results will be shown in Section 5.

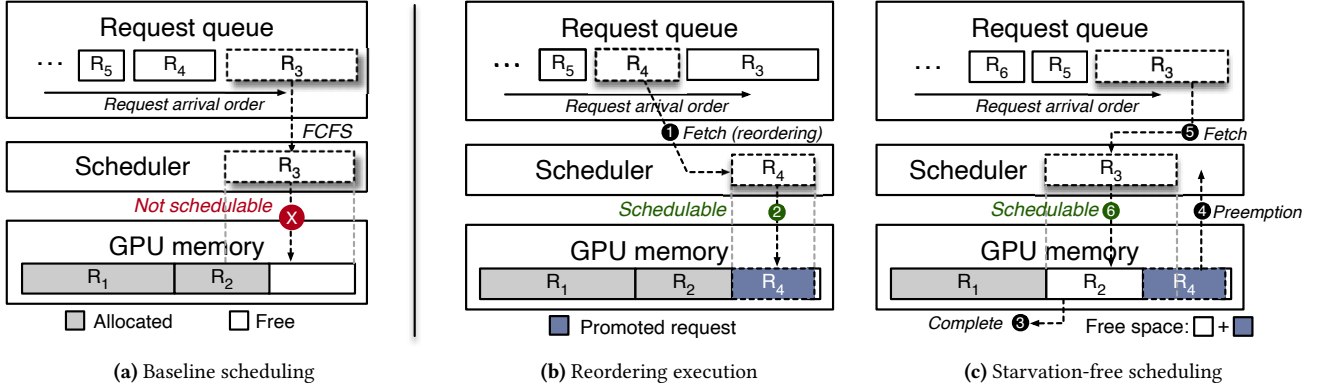


Figure 8. Baseline scheduling (a) and proposed scheduling (b and c)

us to preempt the promoted requests to schedule deferred requests without significantly increasing the waiting time.

Figure 8c depicts how our scheduler prevents the starvation problem for the deferred request (R_3). ③ Suppose R_2 is completed. Then, the free space ($R_2 + R_4$) becomes sufficient to serve the long prompt (R_3), in which case ④ we preempt the temporarily promoted request (R_4), ⑤ thereby facilitating that the deferred request (R_3) is promptly scheduled.

As soon as one of the preceding requests (R_1 or R_3) completes, the preempted request (R_4) is immediately resumed. At this point, we restore the copy of the KVs for R_4 from host memory to GPU memory. Note that our FlashGen-Cache helps us to avoid the recomputation phase and minimize the restoration overhead.

With FlashGen-Sched, we can opportunistically reclaim the space used for caching to promote awaiting requests. As a result, we can shorten the average latency of token generation and improve the throughput per token by effectively increasing the batch size. More importantly, our scheduling method does not depend on our caching technique. This can be solely used to minimize the waiting time for long prompts.

5 Evaluation

5.1 Experimental Setup

Environment: We use PyTorch v2.3 [5] and CUDA 12.1 [23]. Our testbed is an Azure instance, Standard_NC48ads_A100_v4, which provides two NVIDIA A100 (80GB) GPUs. This instance has 440 GB of host DRAM. As default, we allocate about 50% of memory (224GB) for caching history KVs. We configure 2 NVMe SSDs (960GB for each) with a RAID-0 volume to improve the read and write bandwidth.

Models: We use the popular LLM models, OPT [38] and Llama-2 [32]. The Llama-2 70B model uses grouped-query attention (GQA) while the rest of the models are equipped with multi-head attention (MHA). The OPT 13B and Llama-2 13B models are evaluated in a single GPU, while the OPT

30B and Llama-2 70B models are evaluated under two GPUs using tensor parallelism [30]. All the models are FP-16 formatted versions [19] and we extend the maximum sequence length of the models to 16k to evaluate multi-turn prompts.

Comparisons: We evaluate the effectiveness of our techniques in comparison to vLLM and a concurrent work, *Cache-Attention* [12]. We explore our three design options: memory-aware scheduling (FlashGen-Sched), multi-level KV caching (FlashGen-Cache), and the integration of the two (FlashGen). Although FlashGen-Cache is similar to CachedAttention, it has a different capability that opportunistically chooses between recomputing KVs and retrieving them from SSD to minimize the negative impact of SSD involvement. Furthermore, FlashGen-Sched addresses head-of-line blocking caused by amplified prompts in multi-turn dialogues, improving GPU memory utilization—an aspect not covered by CachedAttention.

Attention kernels: Both the baseline and our techniques employ the Flash-Attention [9] and Flash-Decoding [10] techniques for prompt and generation phases, respectively. When restoring KVs from the host memory or SSD, there is no guarantee that memory blocks² pertaining to the previous turns are contiguous in the physical GPU memory. This limitation stems from the attention kernel for prompt phases not being designed with multi-turn scenarios in mind. To deal with non-contiguously stored KVs, we revise the Flash-Attention [9] and Flash-Decoding [10] kernels used for prompt phases and generation phases in vLLM, respectively. This modification is similar to the implementation of the FlashInfer [36] library.

Benchmark: For performance evaluation, we mimic real-world chat scenarios by replaying a real-world chatbot dataset from ShareGPT [27]. We also use two other popular datasets, Alpaca [31] and HumanEval [8]. However, since these two

²Each block is a fixed-size management unit for storing attention keys and values in PagedAttention.

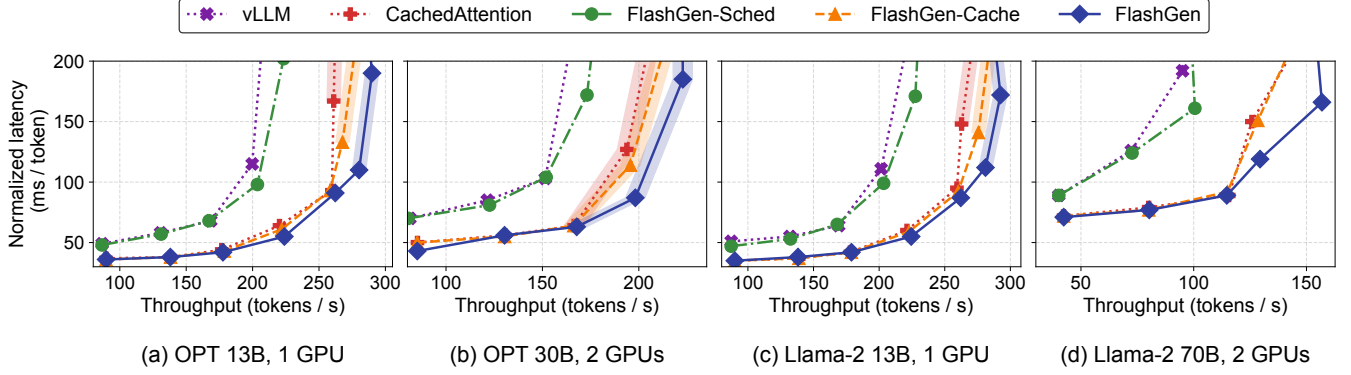


Figure 9. End-to-end latency normalized by the number of generated tokens and throughput in ShareGPT [27] (The shaded points indicate SSD involvement.)

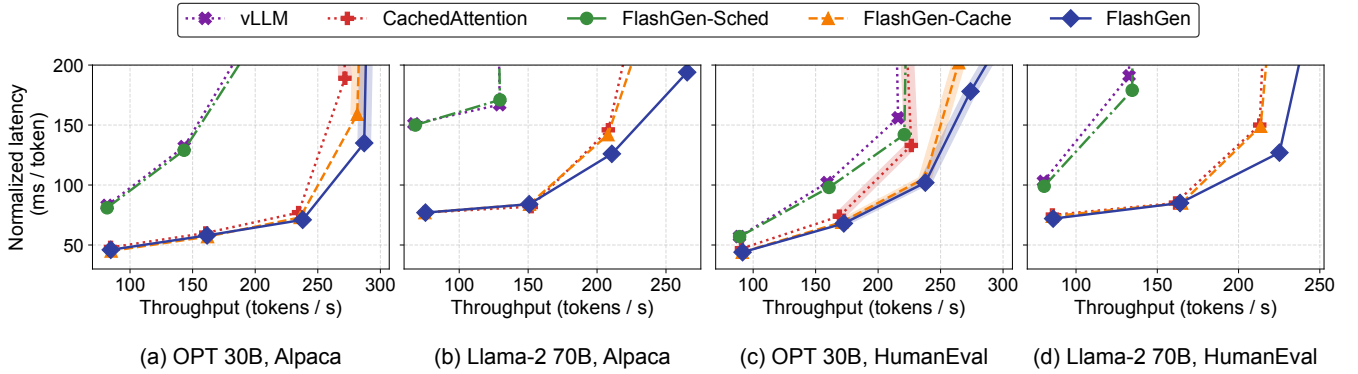


Figure 10. End-to-end latency normalized by the number of generated tokens and throughput in Alpaca [31] and HumanEval [8]

datasets are not comprised of multi-turn conversations, we configure them to have multi-turn characteristics by applying the turn distribution of ShareGPT while preserving their input and output length.

Each client acts as a load generator, sending a request (prompt) and receiving a response (generation). Once a turn is completed, a client sends the next prompt. The interval between turns depends on the length of the prompt and the generation. According to Rayner’s work, humans can read average 300 words per minute [26]. We set the time per token to $1 \text{ minute} / 300 \text{ words} = 200 \text{ ms}$ in our evaluation. Each interval is the sum of the number of prompt tokens and previous output tokens, then multiplied by 200ms. After completing a session, the client executes the next session. By varying the number of clients, we adjust the workloads to the LLM serving framework.

5.2 End-to-end Latency and Throughput

We measure the end-to-end token latency and throughput for OPT and Llama-2 models by increasing the number of clients (loads) to the serving framework. As in previous studies [17, 37], we present normalized latency where the end-to-end latency is divided by the number of output tokens.

Figure 9 exhibits the normalized token latency (y-axis) and token throughput (x-axis) as the number of clients increases. The ShareGPT dataset is used for generating prompts and outputs. Overall, our two techniques, FlashGen-Cache and FlashGen-Sched, can improve both latency and throughput, compared to vLLM, in the four different models. Compared to CachedAttention, our FlashGen-Cache performs better as the load increases, as FlashGen-Cache can dynamically choose between recomputation and retrieving historical KVs from SSD. The shaded points in the figure indicate SSD involvement. The integration version, FlashGen, improves performance further compared to individual schemes. As FlashGen-Sched increases the GPU memory utilization, FlashGen-Cache can handle more requests, leading to throughput improvement.

Figure 10 presents the latency and throughput results for two different datasets, Alpaca and HumanEval with two selected models, OPT 30B and Llama-2 70B, running on two GPUs. Regardless of the datasets or models, our FlashGen significantly outperforms the baseline performance, and our two techniques contribute to performance improvement by reducing the recomputation cost and increasing GPU memory utilization. In the Llama-2 70B model, FlashGen-Cache

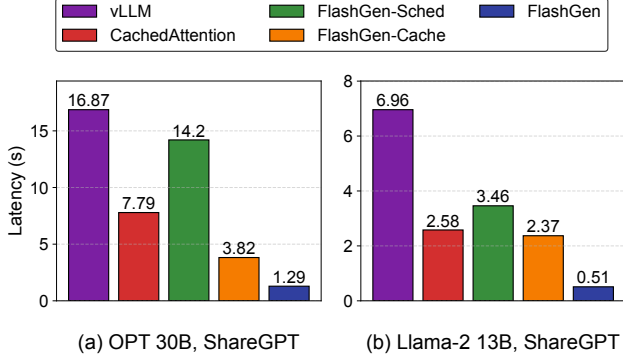


Figure 11. P95 time to first token (TTFT) when serving OPT 30B and Llama-2 13B for ShareGPT

shows a similar performance to CachedAttention. This is because the host memory is sufficient to keep all history KVs. They do not utilize SSD in that case. However, our integrated version, FlashGen, outperforms CachedAttention with the increased GPU memory utilization.

As the request load increases, our FlashGen-Sched demonstrates a trend of improving performance. In the OPT 30B model shown in Figure 9b, our scheduling technique shows a latency improvement of approximately 1.28 $\times$ while providing a similar throughput of around 165~175 tokens per second. Under light loads, however, the performance benefit from FlashGen-Sched is marginal because the head-of-line blocking problem is unlikely to occur. FlashGen-Sched is relatively more beneficial in the limited GPU memory. This is because long prompts are more likely to be in the request queue when the available GPU memory is insufficient.

With FlashGen-Cache, the performance improvement is remarkable in all the models. The main performance benefit comes from replacing the recomputation of history KVs with caching. In Figure 9b, when generating 166 tokens per second, the generation latency is around 64 ms on average. On the other hand, the baseline can only achieve 81 tokens per second in a similar latency boundary. Our integrated version, FlashGen, presents the best performance in terms of both latency and throughput in all cases as the load pressure on the server increases. In the 100ms latency boundary, for the OPT 13B and 30B models shown in Figure 9, FlashGen exhibits around 1.56~1.63 $\times$ throughput over the baseline. The Llama-2 13B and 70B models present about 1.55 $\times$ and 2.85 $\times$ throughput improvement, respectively.

To evaluate the responsiveness of our techniques, Figure 11 presents the tail latency (P95) for time to first token (TTFT) when serving OPT 30B and Llama-2 13B for the ShareGPT dataset with a load of around 1 request per second on average. Compared to the baseline, our scheduling technique reduces TTFT by 16% and 50% for each case. With our caching technique, the latency is further reduced by 77% and 66%. When integrating the caching and scheduling techniques, the responsiveness can be drastically improved.

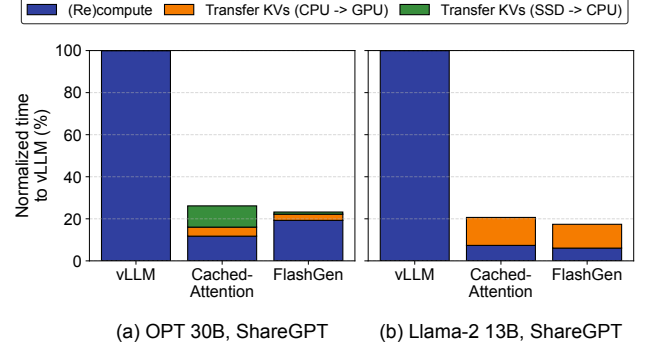


Figure 12. Time breakdown for processing prompt phases

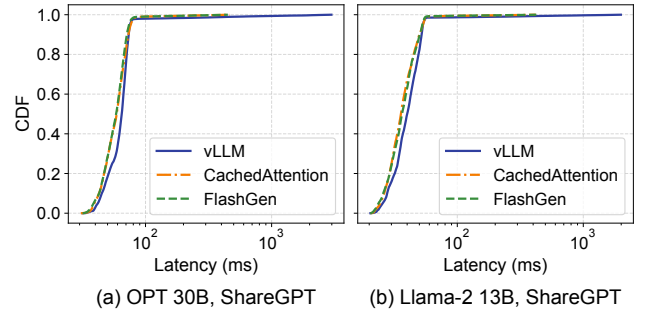


Figure 13. CDF of time per output token (TPOT) from Figure 11's OPT 30B and Llama-2 13B models

To understand the performance benefit of FlashGen over CachedAttention, we decompose the time for processing prompt phases into compute and KV transfer in Figure 12. The compute region includes processing input tokens of current turns and history tokens of previous turns. The transfer region presents two cases: transferring KVs from host to GPU and SSD to host. For OPT 30B, FlashGen spends more time on (re)computing prompts than CachedAttention but requires less time for transferring KVs from SSD to host memory. This is because FlashGen opportunistically retrieves history KVs of previous turns if the transfer time can be (partially) overlapped with the computation. As a result, FlashGen performs 1.13 $\times$ better than CachedAttention. For Llama-2 13B, we do not observe the performance benefit of dynamically selecting either recomputation or retrieving KVs from SSD because all history KVs are kept in the host memory. Nevertheless, FlashGen performs 1.19 $\times$ better than CachedAttention. FlashGen utilizes the GPU memory more effectively than CachedAttention by employing GPU caching, which reduces the time for transferring KVs from host memory to GPU.

Also, we measure the changes in time per output token (TPOT). Figure 13 provides the cumulative distribution function (CDF) for TPOT extracted from the evaluation presented in Figure 11. As FlashGen alleviates the recomputation cost with caching, it substantially reduces the token generation

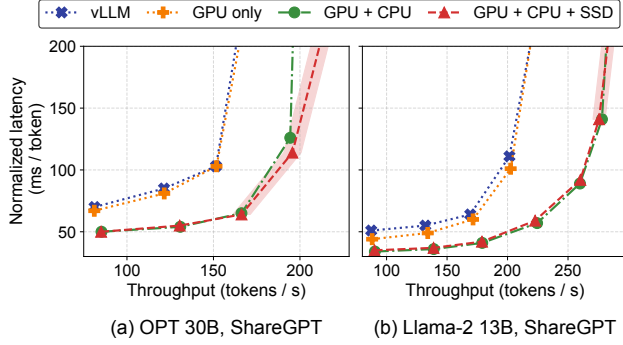


Figure 14. Performance comparison of KV caching strategies for OPT 30B and Llama-2 13B

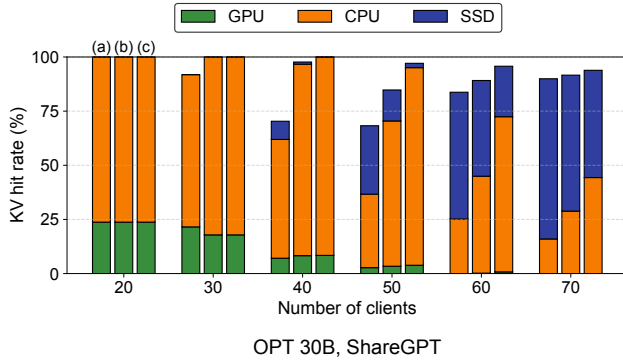


Figure 15. Hit rate for history KV in three different CPU memory sizes (a: 168GB, b: 224GB, c: 280GB)

latency. We do not observe that the KV transmission negatively affects the iteration time while vLLM exhibits a long tail distribution. For OPT 30B, the P99 latency of output token generation is 103ms with FlashGen, compared to 608ms with vLLM. We observe a similar behavior in Llama-2 13B. Compared to CachedAttention, TPOT shows a similar performance result in both models.

5.3 Analysis of Multi-level Caching Strategies

To analyze the performance gain in FlashGen-Cache, we measure performance for each caching option separately. We select two models, OPT 30B and Llama-2 13B, from Figure 9 and the evaluation is done with ShareGPT. Figure 14 shows the performance comparison of our three design configurations: (1) GPU-only, (2) GPU+CPU, and (3) GPU+CPU+SSD. At low loads, the GPU-only design shows improved performance over the baseline. However, as the request load increases, the performance improvement becomes marginal. This is because the available GPU memory for caching history KVs is shrunk due to the increased number of ongoing requests. Consequently, the hit rate for history KVs in the GPU memory decreases.

Figure 15 decomposes the hit rate of history KVs. The remaining portion indicates recomputation, which is considered cache misses. Note that even though the SSD capacity is

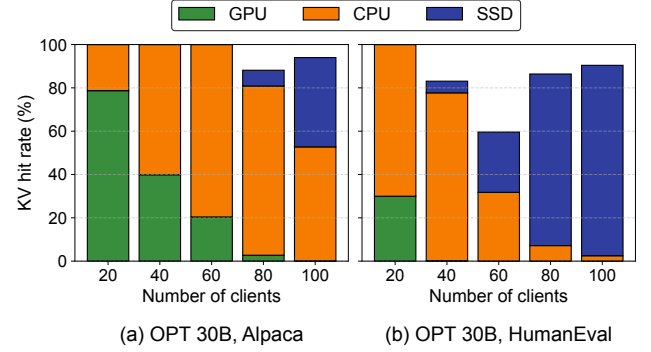


Figure 16. History KV hit rate for Alpaca and HumanEval evaluated in Figure 10

sufficient to keep all previous KVs, FlashGen can select the recomputation option. We also evaluate the OPT 30B model across three different CPU memory sizes for a sensitivity study. The CPU cache size is 3, 4, and 5 times the size of the model: 168GB, 224GB, and 280GB, respectively. The GPU portion steadily decreases as the number of clients (loads) increases. Even with 20 clients, the GPU cache portion falls below 25%. With more than 40 clients, this indicates that the GPU-only design becomes ineffective. When enabling caching history KVs in the host memory, the GPU+CPU design can further improve latency and throughput shown in Figure 14. Nevertheless, the portion retrieving KVs from the host memory also gradually decreases as the load increases. As the size of CPU memory decreases, we need to retrieve the required KVs more from SSD or recompute them. With the integration of SSD (GPU+CPU+SSD), the latency and throughput performance are significantly improved by efficiently replacing the recomputation cost with the KV restoration.

Similarly, in the Llama-2 13B model (not shown in a figure), the GPU cache hit rate is also drastically impacted by increasing the number of clients. Since the size of attention KVs for Llama-2 13B is relatively smaller than that of OPT 30B, the SSD is rarely utilized. As a result, the GPU+CPU and GPU+CPU+SSD designs present similar performance.

Figure 16 presents the KV hit rate for the other two datasets: Alpaca and HumanEval. Each data point is extracted from the evaluation of Figure 10. The GPU+CPU+SSD configuration achieves a higher cache hit rate compared to the other setups. In HumanEval, the GPU+CPU+SSD hit rate decreases steadily as load increases but begins to rise from 80 of clients. Under higher loads, requests are more likely to wait in the queue, providing opportunities to hide the latency of retrieving KVs from SSD. Thus, FlashGen-Cache opts for retrieval over recomputation. In contrast, CachedAttention solely relies on SSD, leading to limited throughput, as shown in Figure 10c.

We also perform a sensitivity study by varying the CPU memory size. Figure 17 presents the end-to-end performance of FlashGen and CachedAttention for three different CPU

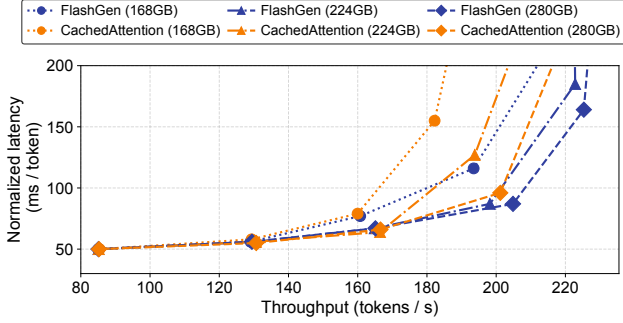


Figure 17. CPU memory sensitivity for OPT 30B

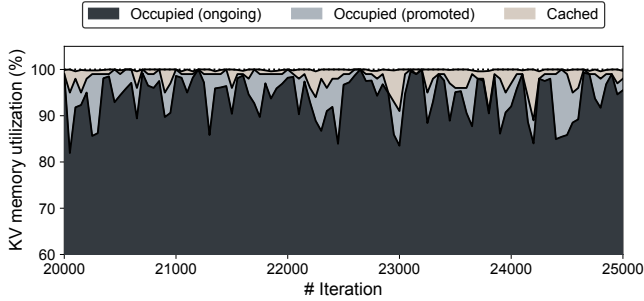


Figure 18. GPU memory utilization while serving an OPT 30B model with the ShareGPT dataset

memory sizes. Generally, the larger the cache, the more requests can be served with low latency. As the load increases, the CPU portion in retrieving KVs decreases inevitably as shown in Figure 15, leading to the performance gap. Also, as the caching space decreases, the performance gap between FlashGen and CachedAttention is remarkable because FlashGen can minimize the SSD involvement.

5.4 Analysis of Request Reordering Technique

In this section, we investigate the performance improvement by FlashGen-Sched. Figure 18 presents the memory utilization by decomposing three regions: (1) *occupied* for ongoing requests, (2) *occupied* for promoting requests, and (3) *cached* for caching history KVs. We extract the utilization data from Figure 9b with a load of around 1 request per second on average. The occupied-promoted region indicates that our scheduling technique utilizes GPU memory for executing re-ordered requests instead of caching history KVs. If there are no reordering opportunities, the memory is left available for caching KVs. For other workloads, we can achieve effective memory utilization of more than 98% on average, which increased by 10% compared to vLLM. As we effectively utilize the GPU memory space, we can dispatch additional requests, leading to larger batch sizes on average. Table 2 shows the increased average number of batched requests normalized to vLLM. In OPT 30B, FlashGen shows 1.15 $\times$ larger batch size on average.

Figure 9				
	OPT 13B	OPT 30B	Llama-2 13B	Llama-2 70B
FlashGen	1.15 $\times$	1.15 $\times$	1.06 $\times$	1.06 $\times$

Table 2. Increase in the average number of batched requests normalized to vLLM

5.5 Request Reordering with Increased Context

Additionally, we evaluate FlashGen-Sched with a synthetic dataset that follows a similar distribution of ShareGPT. For the synthetic dataset, we generate three traces by increasing the context length. The first trace includes the requests with prompt length ranging from 4 to 16k, and the output length ranging from 32 to 1024. We then scale these lengths proportionally: the maximum prompt length increases to 24k and 32k, while the maximum output length scales to 1536 and 2048, respectively. Figure 19 shows improved throughput and GPU memory utilization over the baseline for each trace. For the maximum prompt length of 16k, vLLM shows an average memory utilization of 92%, while FlashGen-Sched reaches 99%. At the maximum prompt length of 32k, the memory utilization in vLLM drops to 78%, whereas FlashGen-Sched maintains it at 97%. In this case, FlashGen-Sched demonstrates a 1.17 $\times$ increase in throughput compared to vLLM.

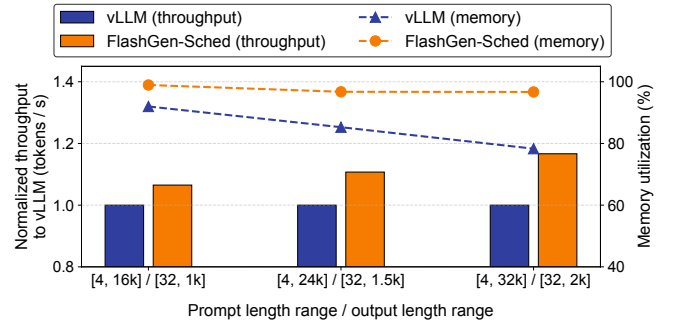


Figure 19. Token throughput and GPU memory utilization according to the context length distribution

6 Related Work

There have been significant efforts in optimizing LLM inferences. We summarize a couple of important and related techniques to our study. Most of the following techniques are employed in LLM serving frameworks such as Orca, TensorRT-LLM, vLLM, and TGI.

KV reuse: There are several studies optimizing multi-turn dialogues. SGLang [39] and ChunkAttention [35] focused on efficiently managing KVs by sharing common prefixes across multiple requests. However, they store KVs only in GPU memory, without utilizing host memory or SSDs. Cache-Attention [12] also proposed storing history KVs in both host memory and SSDs. However, unlike CachedAttention,

FlashGen-Cache does not always retrieve history KVs from SSD. In case of that the latency of retrieving KVs from SSD is higher than recomputation, FlashGen-Cache selects the recomputation option dynamically.

Beyond a single server, FlashGen can be extended to include remote memory in the caching hierarchy such as MemServe [14] and InfiniteLLM [18]. We expect that RDMA-enabled memory, being faster than SSD, has the potential to improve performance by reducing storage access times.

Scheduling: Since each LLM inference request has a different number of output tokens, batching multiple requests together is considered harmful. When a request in a batch is completed, it cannot be returned immediately if another request in the same batch is still processing, leading to unnecessary inference latency and GPU resource wastage. To tackle this problem, iteration-level scheduling has been widely adopted in LLM serving systems [15, 17, 33, 37]. When handling multiple requests in batches, it allows for individual requests to receive immediate responses upon completion, irrespective of the processing status of other requests within the same batch. Furthermore, this approach permits the initiation of a new request in the subsequent iteration, eliminating the need to wait for the completion of the previous batch.

Recently, FastGen [20] and Sarathi-Serve [2] addressed an inefficiency issue related to handling long prompts in the iteration-level scheduling. In this scheduling approach, the execution time of a batch involves two phases: prompt and generation. When the prompt length increases, there is a delay even in processing requests during the generation phase. This is because the requests belonging to the same batch are processed together. Sarathi-Serve introduced a method of splitting a long prompt into smaller chunks and distributing them across multiple iterations. This approach serves to decrease the processing time of a batch, mitigating the adverse effects on generation phases.

Although there are several efforts in scheduling, none of the prior studies address the head-of-line blocking problem incurred by amplified prompts in multi-turn dialogues, which reduces GPU memory utilization. In contrast, FlashGen-Sched goes a step further by optimizing GPU memory utilization through effective scheduling.

Memory optimizations: Kwon et al. introduced a memory management technique, called PagedAttention, to maximize GPU memory utilization [17]. The proposed technique employs the classic virtual memory concept to manage the memory space for attention KVs. Traditionally, the memory for attention KVs is allocated equal to the model's maximum number of tokens for a given request since the number of output tokens is unknown. In contrast, PagedAttention divides memory into fixed-size blocks and gradually allocates them, minimizing memory wastage.

FlexGen investigated offloading techniques for both models and attention KVs to host memory and storage [29].

Our approach, which leverages host resources, aligns with their strategy. However, while FlexGen aimed to maximize throughput at the expense of latency, our system is designed to achieve high throughput for multi-turn services while in a similar latency boundary.

Attention optimizations: Dao et al. optimized the attention kernel, called Flash-Attention, tailored for modern GPU memory hierarchy [9]. By effectively utilizing GPU on-chip SRAM with cache tiling, it reduces the number of global memory accesses. They also introduced Flash-Decoding to accelerate decoding phases by parallelizing long sequence processing [10]. FlashInfer provides such high-performance GPU kernels as a library [36].

Meanwhile, there are several efforts to reduce the computation and memory cost of the standard Multi-Head Attention (MHA) method by reducing the dimensions from the K and V values in the transformer architecture. Multi-Query Attention (MQA) proposed to reduce the number of KV heads to 1 [28]. Although it can reduce the cost significantly, it has been known to incur a negative impact on accuracy. Recently, Llama-2 [32] introduced the Grouped Query Attention (GQA) [3] technique. It makes up the downside of MQA by grouping multiple query heads to share the same KV heads. Relatively small LLMs such as Llama-2 7B and 13B utilize MHA while more than 34B models adopt GQA.

7 Conclusions

This paper proposed *FlashGen* to accelerate multi-turn dialogues by efficiently utilizing the compute and memory resources of GPUs and the host hardware. We analyzed that state-of-the-art LLM frameworks are inefficient in serving multi-turn conversations and identified two sources of limiting performance. Our multi-level caching technique could preserve attention keys and values in GPU, CPU, and SSD so that it minimizes the recomputation phase for prior attention KVs in multi-turn scenarios. In addition to that, our request reordering technique could effectively utilize GPU memory, minimizing the waste of GPU memory capacity.

As the number of turns per session increases (i.e., long conversations), the context size for attention KVs increases proportionally, and the prompt length is further amplified. In line with this trend, we anticipate that *FlashGen-Cache* and *FlashGen-Sched* will become more important in handling these expanded contexts efficiently.

Acknowledgments

We thank the anonymous reviewers for providing helpful feedback and suggestions to improve our work. This work was supported by the National Research Foundation of Korea (NRF) (RS-2023-00211901) and the Institute of Information & communications Technology Planning & Evaluation (IITP) (RS-2024-00396013), funded by the Korea government (MSIT). Jeongseob Ahn is the corresponding author of this paper.

References

- [1] Megha Agarwal, Asfandiyar Qureshi, Nikhil Sardana, Linden Li, Julian Quevedo, and Daya Khudia. 2023. LLM Inference Performance Engineering: Best Practices. <https://www.databricks.com/blog/llm-inference-performance-engineering-best-practices>.
- [2] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.
- [3] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebron, and Sumit Sanghai. 2023. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
- [4] Reza Yazdani Aminabadi, Samyam Rajbhandari, Ammar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Olatunji Ruwase, Shaden Smith, Minjia Zhang, Jeff Rasley, and Yuxiong He. 2022. DeepSpeed-Inference: Enabling Efficient Inference of Transformer Models at Unprecedented Scale. In *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis (SC)*.
- [5] Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burovski, Geeta Chauhan, Anjali Chourdia, Will Constable, Alban Desmaison, Zachary DeVito, Elias Ellison, Will Feng, Jiong Gong, Michael Gschwind, Brian Hirsh, Sherlock Huang, Kshiteej Kalam-barkar, Laurent Kirsch, Michael Lazos, Mario Lezcano, Yanbo Liang, Jason Liang, Yinghai Lu, CK Luk, Bert Maher, Yunjie Pan, Christian Puhersch, Matthias Reso, Mark Saroufim, Marcos Yukio Siraichi, Helen Suk, Michael Suo, Phil Tillet, Eikan Wang, Xiaodong Wang, William Wen, Shunting Zhang, Xu Zhao, Keren Zhou, Richard Zou, Ajit Mathews, Gregory Chanan, Peng Wu, and Soumith Chintala. 2024. PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation. In *29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.
- [6] Zhihao Bai, Zhen Zhang, Yibo Zhu, and Xin Jin. 2020. PipeSwitch: Fast Pipelined Context Switching for Deep Learning Applications. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.
- [7] Iz Beltagy, Matthew E. Peters, and Arman Cohan. 2020. Longformer: The Long-Document Transformer. arXiv:2004.05150 [cs.CL]
- [8] Mark Chen, Jerry Twarek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating Large Language Models Trained on Code. arXiv:2107.03374 [cs.LG]
- [9] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [10] Tri Dao, Daniel Haziza, Francisco Massa, and Grigory Sizov. 2023. Flash-Decoding for long-context inference. <https://pytorch.org/blog/flash-decoding/>.
- [11] FastChat. 2023. <https://github.com/lm-sys/FastChat>.
- [12] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. 2024. Cost-Efficient Large Language Model Serving for Multi-turn Conversations with CachedAttention. In *2024 USENIX Annual Technical Conference (ATC)*.
- [13] Google. 2023. <https://bard.google.com/>.
- [14] Cunhen Hu, Heyang Huang, Junhao Hu, Jiang Xu, Xusheng Chen, Tao Xie, Chenxi Wang, Sa Wang, Yungang Bao, Ninghui Sun, and Yizhou Shan. 2024. MemServe: Context Caching for Disaggregated LLM Serving with Elastic Memory Pool. arXiv:2406.17565 [cs.DC]
- [15] HuggingFace. 2023. <https://github.com/huggingface/text-generation-inference>.
- [16] Sam Ade Jacobs, Masahiro Tanaka, Chengming Zhang, Minjia Zhang, Shuaiwen Leon Song, Samyam Rajbhandari, and Yuxiong He. 2023. DeepSpeed Ulysses: System Optimizations for Enabling Training of Extreme Long Sequence Transformer Models. arXiv:2309.14509 [cs.LG]
- [17] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *29th ACM Symposium on Operating Systems Principles (SOSP)*.
- [18] Bin Lin, Chen Zhang, Tao Peng, Hanyu Zhao, Wencong Xiao, Minmin Sun, Anmin Liu, Zhipeng Zhang, Lanbo Li, Xiafei Qiu, Shen Li, Zhigang Ji, Tao Xie, Yong Li, and Wei Lin. 2024. Infinite-LLM: Efficient LLM Service for Long Context with DistAttention and Distributed KVCache. arXiv:2401.02669 [cs.DC]
- [19] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. 2018. Mixed Precision Training. In *International Conference on Learning Representations (ICLR)*.
- [20] Microsoft. 2023. DeepSpeed-FastGen: High-throughput Text Generation for LLMs via MII and DeepSpeed-Inference. <https://github.com/microsoft/DeepSpeed/tree/master/blogs/deepspeed-fastgen>.
- [21] NVIDIA. 2019. GPUDirect Storage: A Direct Path Between Storage and GPU Memory. <https://developer.nvidia.com/blog/gpudirect-storage>.
- [22] NVIDIA. 2024. NVIDIA GPUDirect Storage Benchmarking and Configuration Guide. <https://docs.nvidia.com/gpudirect-storage/configuration-guide/index.html>.
- [23] NVIDIA, Péter Vingelmann, and Frank H.P. Fitzek. 2020. CUDA, release: 12.1. <https://developer.nvidia.com/cuda-toolkit>.
- [24] OpenAI. 2023. <https://openai.com/blog/new-models-and-developer-products-announced-at-devday>.
- [25] OpenAI. 2023. <https://openai.com/blog/chatgpt>.
- [26] Keith Rayner. 1978. Eye Movements in Reading and Information Processing. *Psychological bulletin* 85 (05 1978), 618–60. <https://doi.org/10.1037/0033-2909.85.3.618>
- [27] ShareGPT. 2023. https://huggingface.co/datasets/anon8231489123/ShareGPT_Vicuna_unfiltered/tree/main.
- [28] Noam Shazeer. 2019. Fast Transformer Decoding: One Write-Head is All You Need. arXiv:1911.02150 [cs.NE]
- [29] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. 2023. FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*.
- [30] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. CoRR abs/1909.08053 (2019). arXiv:1909.08053
- [31] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. 2023. Stanford Alpaca: An Instruction-following LLaMA model. https://github.com/tatsu-lab/stanford_alpaca.

- [32] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. LLaMA: Open and Efficient Foundation Language Models. arXiv:2302.13971 [cs.CL]
- [33] Neal Vaidya, Nick Comly, Joe DeLaere, Ankit Patal, and Fred Oh. 2023. NVIDIA TensorRT-LLM Supercharges Large Language Model Inference on NVIDIA H100 GPUs. <https://developer.nvidia.com/blog/nvidia-tensorrt-llm-supercharges-large-language-model-inference-on-nvidia-h100-gpus/>.
- [34] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [35] Lu Ye, Ze Tao, Yong Huang, and Yang Li. 2024. ChunkAttention: Efficient Self-Attention with Prefix-Aware KV Cache and Two-Phase Partition. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*.
- [36] Zihao Ye, Lequn Chen, Ruihang Lai, Yilong Zhao, Size Zheng, Junru Shao, Bohan Hou, Hongyi Jin, Yifei Zuo, Liangsheng Yin, Tianqi Chen, and Luis Ceze. 2024. Accelerating Self-Attentions for LLM Serving with FlashInfer. <https://flashinfer.ai/2024/02/02/introduce-flashinfer.html>
- [37] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.
- [38] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, Todor Mihaylov, Myle Ott, Sam Shleifer, Kurt Shuster, Daniel Simig, Punit Singh Koura, Anjali Sridhar, Tianlu Wang, and Luke Zettlemoyer. 2022. OPT: Open Pre-trained Transformer Language Models. arXiv:2205.01068 [cs.CL]
- [39] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2024. SGLang: Efficient Execution of Structured Language Model Programs. arXiv:2312.07104 [cs.AI]